ACADEMY OF SKILL DEVELOPMENT

Major Project Report

ASD

SMART HOSPITAL MANAGEMENT SYSTEM

A report prepared in accordance with the prescribed project report standards

Submitted By
Shreyansh Verma
Tavis Mariageorge James

Project Guide
Mr. Subhojit Santra

Department / Program : Major Project Submission

Academic Session : 2025 - 2026

Prepared for academic evaluation and final submission in PDF form

New report generated from project instructions, sample format, and the prior draft report.

CERTIFICATE

This is to certify that the project report entitled "Smart Hospital Management System" is a bona fide work carried out by Shreyansh Verma and Tavis Mariageorge James under the guidance and supervision of the undersigned during the academic session 2025 - 2026. The work presented in this report has been completed in accordance with the project report standards prescribed for major project submission.

The report demonstrates the analysis, design, implementation, testing, and evaluation of a web-based Smart Hospital Management System developed using the MERN stack and MVC architecture. To the best of our knowledge, the contents of this report are original and have not been submitted, either in full or in part, for the award of any other degree, diploma, or certificate.

The project fulfills the academic requirements for final submission and is recommended for evaluation.

Project Guide	**Head of Department**	**Dean / Principal**
_____	_____	_____

Date: _____ Place: _____

DECLARATION

We hereby declare that the project report entitled "Smart Hospital Management System" submitted for academic evaluation is an original work carried out by us under the guidance of the project mentor. The project has been designed and implemented using the MERN stack for the purpose of demonstrating a digital solution to manual hospital management.

The matter embodied in this report has not been copied from any earlier submission, and no part of this report has been submitted to any other institution or university for the award of any degree, diploma, or certificate. Whenever the ideas, findings, or documentation of other authors have been referred to, proper acknowledgement has been incorporated in the references section.

We understand that any instance of plagiarism or misrepresentation may lead to academic action as per institutional rules.

Shreyansh Verma	Tavis Mariageorge James
Student Signature	Student Signature

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Mr. Subhojit Santra, for his valuable guidance, patient encouragement, and constructive suggestions throughout the development of this project. His support helped us refine both the technical implementation and the academic presentation of the Smart Hospital Management System.

We also thank the faculty members and academic coordinators of the Academy of Skill Development for providing the learning environment, project review support, and institutional resources required to complete this work. Their feedback during each stage of development helped us improve the structure, scope, and quality of the report.

We are equally grateful to our friends and peers for their suggestions during design validation and testing. Finally, we thank our families for their constant support, motivation, and encouragement throughout the completion of this major project.

ABSTRACT

The Smart Hospital Management System is a full stack web application developed to address the inefficiencies of manual hospital record management. In many small and medium-sized healthcare institutions, patient registration, appointment handling, prescription management, report storage, order processing, and communication are still carried out manually or through fragmented tools. This creates delays, redundancy, poor coordination, and difficulty in retrieving information when needed.

To solve this problem, the proposed system provides a centralized digital platform built using the MERN stack, namely MongoDB, Express.js, React.js, and Node.js. The backend follows the Model-View-Controller architecture to ensure clear separation of concerns and maintainable code. Core features include user authentication, email OTP verification, role-based access control, patient record management, appointment booking, e-prescription support, report uploads, order processing with cart and Cash on Delivery, real-time chat, notifications, and geo-location map integration.

The system was implemented and validated module by module using seeded dummy data and manual testing. The results show that the proposed system successfully improves data accessibility, reduces paper dependency, streamlines appointment workflows, and strengthens communication between administrators, doctors, and patients. The project forms a practical foundation for future enhancements such as telemedicine, wearable device integration, cloud storage, and online payment support.

TABLE OF CONTENTS

Title Page 1

Certificate 2

Declaration 3

Acknowledgement 4

Abstract 5

Table of Contents 6

List of Figures 8

List of Tables 9

Chapter 1: Introduction 10

1A. Background of the Problem 11

1B. Importance and Relevance 12

1C. Problem Statement 13

1D. Objectives of the Project 14

1E. Scope of the Project 15

Chapter 2: Literature Review 16

2A. Research Papers / Articles Reviewed 17

2B. Summary of Existing Work 18

TABLE OF CONTENTS (CONTINUED)

2C. Comparative Analysis 19

2D. Identification of Research Gap 20

2E. Literature Review Synthesis 21

Chapter 3: System Analysis 22

Chapter 4: System Design 32

Chapter 5: Methodology 43

Chapter 6: Implementation 48

Chapter 7: Results and Discussion 56

Chapter 8: Advantages and Limitations 61

Chapter 9: Conclusion 64

Chapter 10: Future Scope 65

References 66

Appendix A: API Endpoints 68

Appendix B: Dummy Data Samples 69

Appendix C: Source Code Organization 70

LIST OF FIGURES

Figure 1. System Architecture of Smart Hospital Management System 33

Figure 2. Module Design Diagram 34

Figure 3. Data Flow Diagram Level 0 35

Figure 4. Data Flow Diagram Level 1 - Patient 36

Figure 5. Data Flow Diagram Level 1 - Admin and Doctor 37

Figure 6. Use Case Diagram 38

Figure 7. Sequence Diagram for Appointment Booking 39

Figure 8. Schema / Entity Relationship Diagram 40

Figure 9. Authentication Flowchart 41

Figure 10. Order Processing Flowchart 42

Figure 11. Representative Interface Layouts 54

LIST OF TABLES

Table 1. Review of Major Research Papers 17

Table 2. Comparative Analysis of Existing Approaches 19

Table 3. Feasibility Study Summary 26

Table 4. Input and Output Analysis 27

Table 5. Tools and Technologies Used 49

Table 6. Hardware Requirements 50

Table 7. Software Requirements and Dummy Data Summary 51

Table 8. Core API Summary 53

Table 9. System Test Cases 57

Table 10. Expected vs Actual Comparison 59

Table 11. Future Enhancement Roadmap 65

CHAPTER 1: INTRODUCTION

Hospital management is one of the most information-intensive and process-driven domains in the healthcare sector. Every hospital must continuously manage patient registration, doctor scheduling, report storage, prescription handling, billing coordination, and administrative communication. The quality of this management directly affects patient experience, service speed, and record accuracy.

In many hospitals and clinics, particularly at small and medium scale, these activities are still handled through manual registers, disconnected spreadsheets, or isolated software tools. Such an environment creates duplication of work, weak traceability, and delays in the retrieval of patient history.

The Smart Hospital Management System was developed to provide a structured digital alternative. This chapter introduces the project background, need, objectives, and scope.

1A. BACKGROUND OF THE PROBLEM

Healthcare institutions operate in a highly time-sensitive environment where timely access to information is critical. Patient records, medical reports, prescriptions, and appointments must be accessible to the right users at the right time. When these details are managed manually, retrieval becomes dependent on physical files, staff memory, and repetitive data entry.

Traditional hospital workflows were designed around paper forms and desk-based coordination. While such systems may function at a small scale, they quickly become inefficient as the patient load increases. Repeated visits create multiple documents across departments, and each new appointment or report update adds to the complexity of record tracking.

A well-designed hospital management system can unify registration, appointments, communication, and record access through one secure platform.

1B. IMPORTANCE AND RELEVANCE

The proposed project is important because hospital administration directly affects the quality and continuity of care. When appointment records are lost, when reports cannot be found quickly, or when patients are not informed about changes, the outcome is inconvenience for both staff and patients. A digital system reduces this operational friction.

This project is relevant in academic as well as practical terms. Academically, it demonstrates how a complete MERN stack solution can be built using role-based access control, REST APIs, JWT authentication, WebSockets, file uploads, and structured MVC design.

Because healthcare systems increasingly rely on reliable information flow, a hospital management application that combines patient services, staff coordination, and structured records has strong relevance.

1C. PROBLEM STATEMENT

The central problem addressed by this project is the inefficient and fragmented nature of manual hospital record management. Important activities such as patient registration, appointment scheduling, prescription tracking, report handling, and administrative communication are frequently maintained using paperwork or disconnected systems that do not integrate with one another.

As a result, the hospital faces repeated data entry, slow search operations, poor coordination between stakeholders, and limited transparency in patient-facing workflows.

- Duplicate patient records and scattered information across departments
- Manual appointment handling with no instant status visibility
- Limited connectivity between admin, doctor, and patient users
- Weak document handling for reports, prescriptions, and uploads
- No integrated cart, order tracking, or notification system
- Difficulty scaling the process as hospital activity grows

1D. OBJECTIVES OF THE PROJECT

- To replace manual hospital record management with a structured digital platform.
- To create secure authentication using email, password, JWT, and OTP verification.
- To maintain role-based access for administrator, doctor, and patient users.
- To digitize patient profile management and appointment scheduling workflows.
- To support e-prescription handling, report uploads, and notification delivery.
- To provide an order processing module with cart management and Cash on Delivery.
- To enable real-time user communication through an integrated chat system.
- To design the solution in MERN stack with clear MVC architecture and modular code.
- To prepare a foundation that can be extended with telemedicine and wearable integration.

1E. SCOPE OF THE PROJECT

The scope of the Smart Hospital Management System includes the digital management of patient records, appointments, file uploads, medical product ordering, notifications, and user communication within a browser-based application. The project supports three user roles: administrator, doctor, and patient.

Within the current implementation, the project covers user registration and login, OTP verification, admin-side CRUD operations, appointment handling, report upload, order processing with cart and COD, map integration, notifications, and real-time chat.

- In scope: authentication, CRUD, search and filter, chat, notification, upload, map, cart, and order history.
- Partially in scope: e-prescription support and representative billing through order processing.
- Out of scope in the current version: live telemedicine, full online payment gateway, insurance claims, and advanced analytics.
- Deployment target: small to medium hospital or clinic level digital workflow prototype.

CHAPTER 2: LITERATURE REVIEW

A literature review was conducted to understand how hospital information systems have evolved, which implementation challenges are commonly reported, and what design principles support better user adoption. The review focused on hospital information systems, user acceptance, implementation complexity, and web-based information delivery in healthcare environments.

The selected studies show that successful hospital information systems must balance technology, workflow design, usability, organizational support, and evaluation mechanisms. These findings directly informed the design of the proposed Smart Hospital Management System.

2A. RESEARCH PAPERS / ARTICLES REVIEWED

Paper	Year	Core Focus	Key Learning
Bakker and Mol	1983	Foundational HIS structure	Centralized records improve coordination and availability
Reichertz	2006	Past, present, future of HIS	Patient-centered electronic information is essential
Bain and Standing	2009	Hospital management information ecosystem	Information needs extend across operations and decisions
Sligo et al.	2017	Planning and evaluation of HIS	Implementation success depends on organizational and human factors
Handayani et al.	2018	User acceptance review	Usability, support, and trust shape adoption
Khalifa and Alswailem	2015	Acceptance and satisfaction case study	Performance and training strongly affect satisfaction

Table 1. Review of major research papers that informed the proposed system design.

2B. SUMMARY OF EXISTING WORK

The reviewed literature indicates that existing hospital information systems generally focus on registration, billing, electronic medical records, or departmental information exchange. Large enterprise HIS solutions provide comprehensive integration but are expensive, technically demanding, and difficult to customize for small institutions.

At the other end of the spectrum, smaller clinics often depend on isolated solutions such as spreadsheet records, appointment books, or standalone billing tools. These systems may handle one task effectively but do not create a single source of truth for hospital operations.

The literature also shows that user adoption drops when systems are slow, overly complex, or unsupported by staff training.

- Most existing solutions prioritize one department rather than the full patient journey.
- Interoperability and integrated communication remain major challenges in practice.
- Training, performance, and role-specific usability are repeatedly identified as adoption factors.
- Smaller institutions need systems that are simpler, modular, and more affordable to operate.

2C. COMPARATIVE ANALYSIS

Approach	Typical Features	Advantages	Limitations
Manual System	Paper files, phone scheduling	Low initial cost	Slow retrieval, error-prone, no analytics
Isolated Digital Tools	Spreadsheet or desktop modules	Simple to begin with	No central integration, repeated entries
Enterprise HIS	Clinical, admin, billing integration	Comprehensive and powerful	High cost, heavy training, complex deployment
Proposed System	MERN web app with chat, upload, cart, map, OTP	Centralized, modular, scalable, web-based	Prototype scope, future integrations pending

The comparison highlights a clear need for a middle-path solution: one that is more integrated than manual or isolated tools, yet simpler and more accessible than a large enterprise hospital information platform.

Table 2. Comparative analysis of existing approaches versus the proposed web-based system.

2D. IDENTIFICATION OF RESEARCH GAP

- Existing studies emphasize implementation and adoption challenges, but many practical prototypes still leave out communication, cart-based ordering, and unified user engagement.
- Several hospital information systems are either too narrow in function or too heavy for academic-level or small-institution deployment.
- There is limited emphasis on simple full stack architectures that combine secure authentication, real-time communication, uploads, and operational workflows in one accessible system.
- Many reviewed platforms do not focus on a clear educational implementation using MERN and MVC while still demonstrating realistic hospital modules.
- The present project addresses this gap by combining multiple hospital-facing processes into one structured web system with manageable code complexity.

Therefore, the proposed project is positioned as a practical, modular, and educationally strong implementation that responds to both operational needs and software engineering clarity.

2E. LITERATURE REVIEW SYNTHESIS

The literature review collectively shows that successful hospital information systems must be designed around more than data storage. They must also consider staff usability, communication flow, timely access to records, secure authentication, and organizational fit. Systems that ignore these dimensions may be technically functional but operationally weak.

These findings influenced the proposed system in several ways. First, the application uses role-based access so that each stakeholder sees only relevant features. Second, it includes chat and notification capabilities to reduce coordination delays. Third, the solution uses a web-based architecture so that deployment and access remain simple. Finally, it keeps the code modular through MVC so that further integrations can be added without disrupting the foundation.

CHAPTER 3: SYSTEM ANALYSIS

System analysis examines the operational problem in its current environment, identifies weaknesses in the existing workflow, and justifies the proposed solution. In this project, analysis was carried out from the perspective of hospital administrators, doctors, and patients, with emphasis on how information moves across registration, appointments, records, communication, and ordering.

This chapter presents the existing system, its limitations, the proposed solution, feasibility analysis, input-output analysis, software requirement specifications, and the development paradigm applied to the project.

3A. EXISTING SYSTEM

In the existing workflow, patient information is usually recorded manually at a reception desk or stored in isolated local files. Follow-up details are added separately over time, resulting in partial records distributed across files or departments. Staff must search physically or ask other personnel whenever historical information is needed.

Appointments are typically booked by phone or in person, often without a central digital scheduler. This causes overbooking risk, lack of reminders, and difficulty tracking whether an appointment was completed, postponed, or cancelled. Diagnostic reports and prescriptions remain difficult to organize because they exist in paper form or scattered digital copies.

Administrative communication is also fragmented. Patients often need to make repeated calls to ask about appointments or reports, while staff have no instant way to notify patients about changes in order status, consultation timing, or required documents.

3B. LIMITATIONS OF EXISTING SYSTEM

- Searching patient history is slow because records are manual, fragmented, or inconsistently stored.
- Repeated data entry causes duplicate records and inconsistent patient information.
- Appointment management depends heavily on staff availability and manual checking.
- There is no unified communication layer for notifications or real-time conversation.
- Report handling is weak because file uploads and centralized digital storage are absent.
- Administrative oversight is poor due to the lack of a dashboard and searchable analytics.
- Operational growth becomes difficult because manual systems do not scale with patient volume.
- Security and accountability are limited because role-based digital access is not enforced.

These limitations motivated the shift toward a centralized, browser-based, multi-user system with secure access and structured data handling.

3C. PROPOSED SYSTEM

The proposed Smart Hospital Management System is a centralized MERN stack application that integrates multiple hospital operations within a single digital platform. The administrator can manage users, patients, appointments, and orders; doctors can review appointments and communicate with users; and patients can register, verify identity, upload reports, place orders, and interact with the system through a structured dashboard.

The system provides secure authentication, OTP verification, role-based authorization, patient management, appointment tracking, e-prescription support, file uploads, notifications, real-time chat, a basic medical store with cart and COD, and geo-location assistance.

- Web-based access with centralized MongoDB storage
- MVC backend structure for maintainability
- Operational connectivity between admin, doctor, and patient
- Search, filter, upload, order, map, and communication support

3D. FEASIBILITY STUDY

Feasibility Type	Observation	Conclusion
Technical	MERN stack, MongoDB, Socket.IO, and Multer support all required modules.	Technically feasible
Operational	Users can access the system through a browser with role-based dashboards.	Operationally feasible
Economic	Open-source tools reduce development and deployment cost for prototype scale.	Economically feasible
Schedule	Module-wise development fits an academic project timeline.	Time feasible
Scalability	MVC structure and modular routes allow future feature growth.	Scalable with planned enhancements

The feasibility analysis confirms that the proposed system is practical for academic implementation and can serve as a strong prototype for future institutional use.

Table 3. Feasibility study summary for the proposed Smart Hospital Management System.

3E. INPUT AND OUTPUT ANALYSIS

Module	Primary Inputs	Primary Outputs
Authentication	Name, email, password, OTP	User session, verification status
Patient Management	Age, gender, blood group, disease	Structured patient record
Appointments	Doctor, patient, date, reason	Appointment entry and status
Orders	Product selection and quantity	Cart contents, order confirmation, total amount
Upload	Report file	Stored file path and upload acknowledgment
Chat and Notifications	Message text or event trigger	Delivered messages and alert entries

Table 4. Input and output analysis of the core modules.

3F. SOFTWARE REQUIREMENT SPECIFICATION (FUNCTIONAL)

- The system shall allow user registration and secure login.
- The system shall generate and verify email OTP for account verification.
- The system shall enforce role-based access for admin, doctor, and patient users.
- The admin shall perform create, read, update, and delete operations on patient records.
- The system shall support appointment booking, status updates, and e-prescription recording.
- The system shall provide search and filter functionality across major record types.
- The system shall support product selection, cart storage, and order placement through COD.
- The system shall allow report file uploads and store their references.
- The system shall support real-time user-to-user chat.
- The system shall generate notifications for important operational events.

3G. SOFTWARE REQUIREMENT SPECIFICATION (NON-FUNCTIONAL)

- Usability: the interface should remain simple enough for non-technical users.
- Security: passwords must be hashed and private routes must be token protected.
- Performance: normal operations should complete within acceptable demo response time.
- Reliability: main CRUD and order flows should remain stable under regular use.
- Maintainability: backend code should follow MVC and frontend components should stay modular.
- Scalability: additional modules such as telemedicine should be addable later.
- Responsiveness: major pages should display properly on standard desktop and laptop screens.
- Availability: the application should run locally with standard Node.js and MongoDB setup.

3H. SOFTWARE ENGINEERING PARADIGM APPLIED

The project follows an incremental and iterative development approach. Instead of attempting the full application in one step, the system was built module by module: first the authentication layer, then patient and appointment management, then order handling, and finally communication, notifications, and AI-assisted interaction support.

On the architectural side, the backend uses the MVC pattern. Models define MongoDB schemas, controllers handle business logic, and routes connect endpoints to the appropriate controller actions. This separation helps developers debug features, extend modules, and maintain clarity.

The frontend follows a component-oriented structure in React, which supports reuse and keeps stateful logic separated from visual sections.

3I. OVERALL WORKFLOW OF THE SYSTEM

- User creates an account and logs in through secure authentication.
- OTP verification confirms identity when required for account trust.
- Admin manages patient records and assigns appointments to doctors.
- Patient views schedules, uploads reports, and interacts with the system dashboard.
- Medical items can be added to cart; repeated additions increase quantity rather than creating duplicate rows.
- Patient places order through COD and receives a confirmation view with total amount.
- Admin updates order status and can remove delivered orders from history when permitted.
- Chat and notification modules maintain communication between the connected users.

This workflow integrates administrative control and patient self-service into one coordinated web system.

CHAPTER 4: SYSTEM DESIGN

System design translates the analyzed requirements into a concrete technical structure. In this project, design includes the layered architecture of the application, the division of modules, the movement of data between actors and processes, and the schema-level relationships that support persistent storage.

To align with the sample report and project guidelines, this chapter includes the system architecture diagram, data flow diagrams, a use case diagram, a sequence diagram, a schema / ER diagram, and flowcharts for core processes.

4A. SYSTEM ARCHITECTURE DIAGRAM

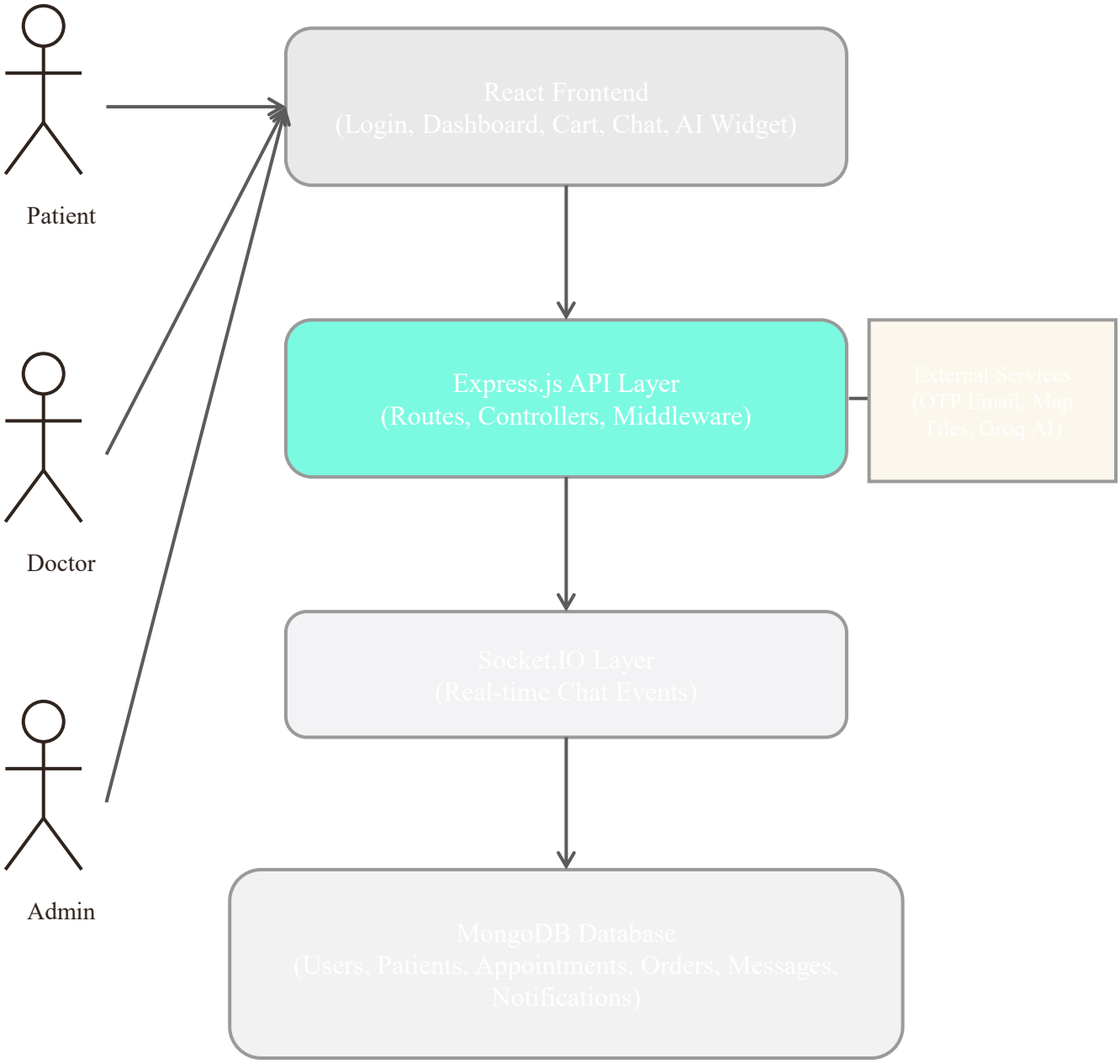

Figure 1. Layered architecture showing users, frontend, API, real-time communication, database, and external services.

4B. MODULE DESIGN

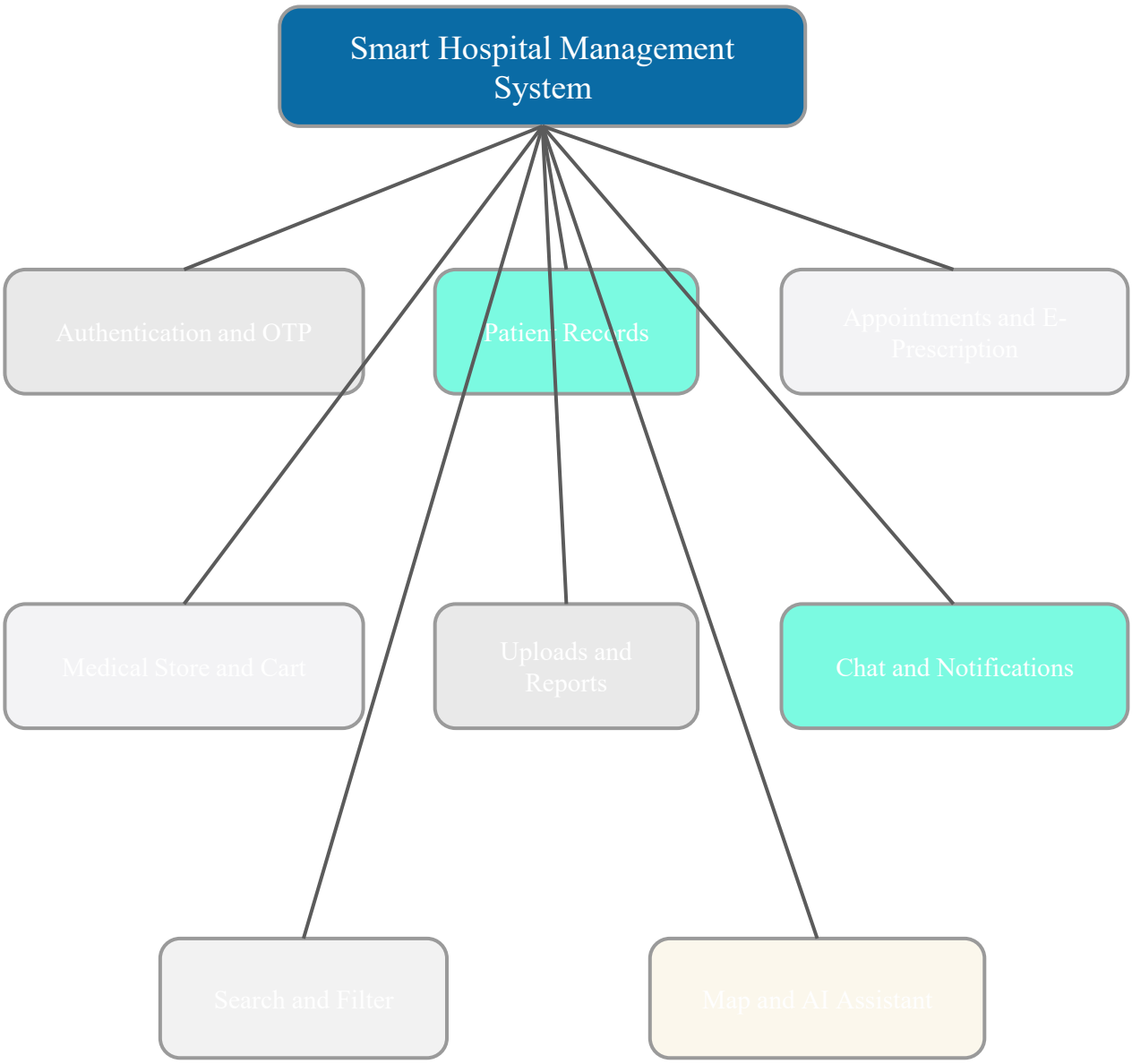

Figure 2. Module-level decomposition of the proposed system.

4C. DATA FLOW DIAGRAM LEVEL 0

Figure 3. Level 0 DFD showing the system as one high-level process connected to major actors.

4D. DATA FLOW DIAGRAM LEVEL 1 - PATIENT

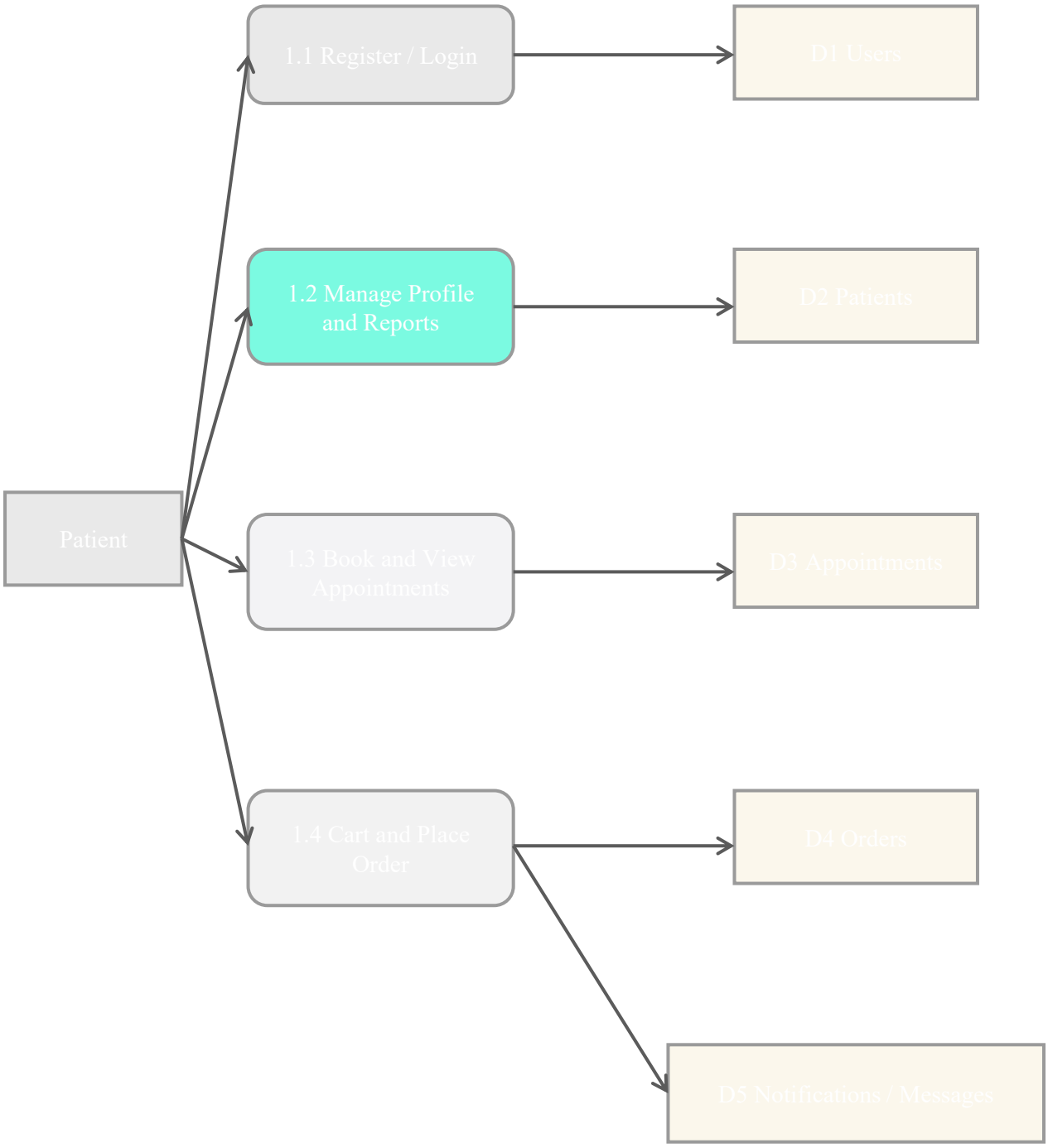

Figure 4. Level 1 DFD describing the main patient-side processes and the connected data stores.

4E. DATA FLOW DIAGRAM LEVEL 1 - ADMIN AND DOCTOR

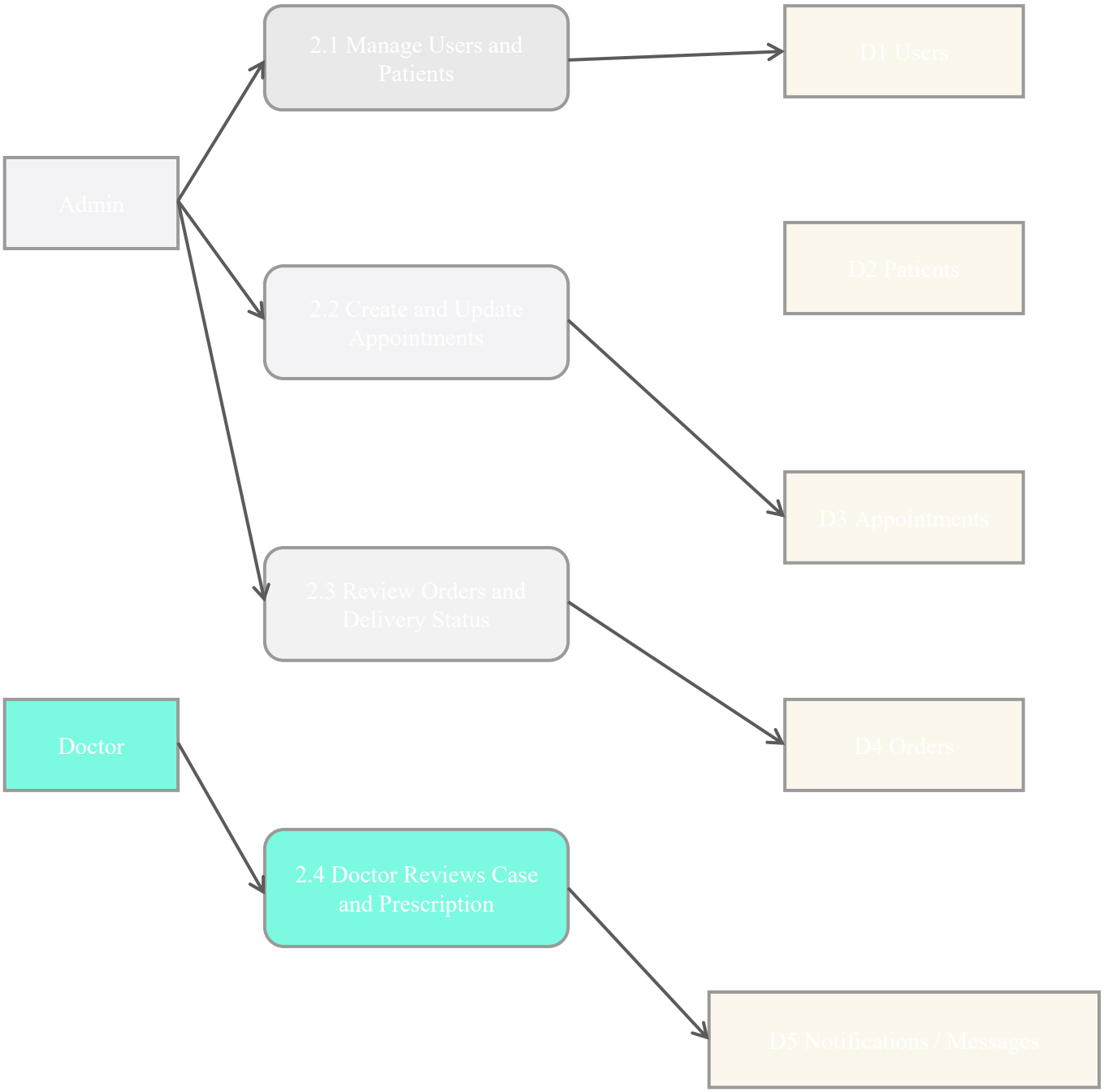

Figure 5. Level 1 DFD describing the admin and doctor operational processes.

4F. USE CASE DIAGRAM

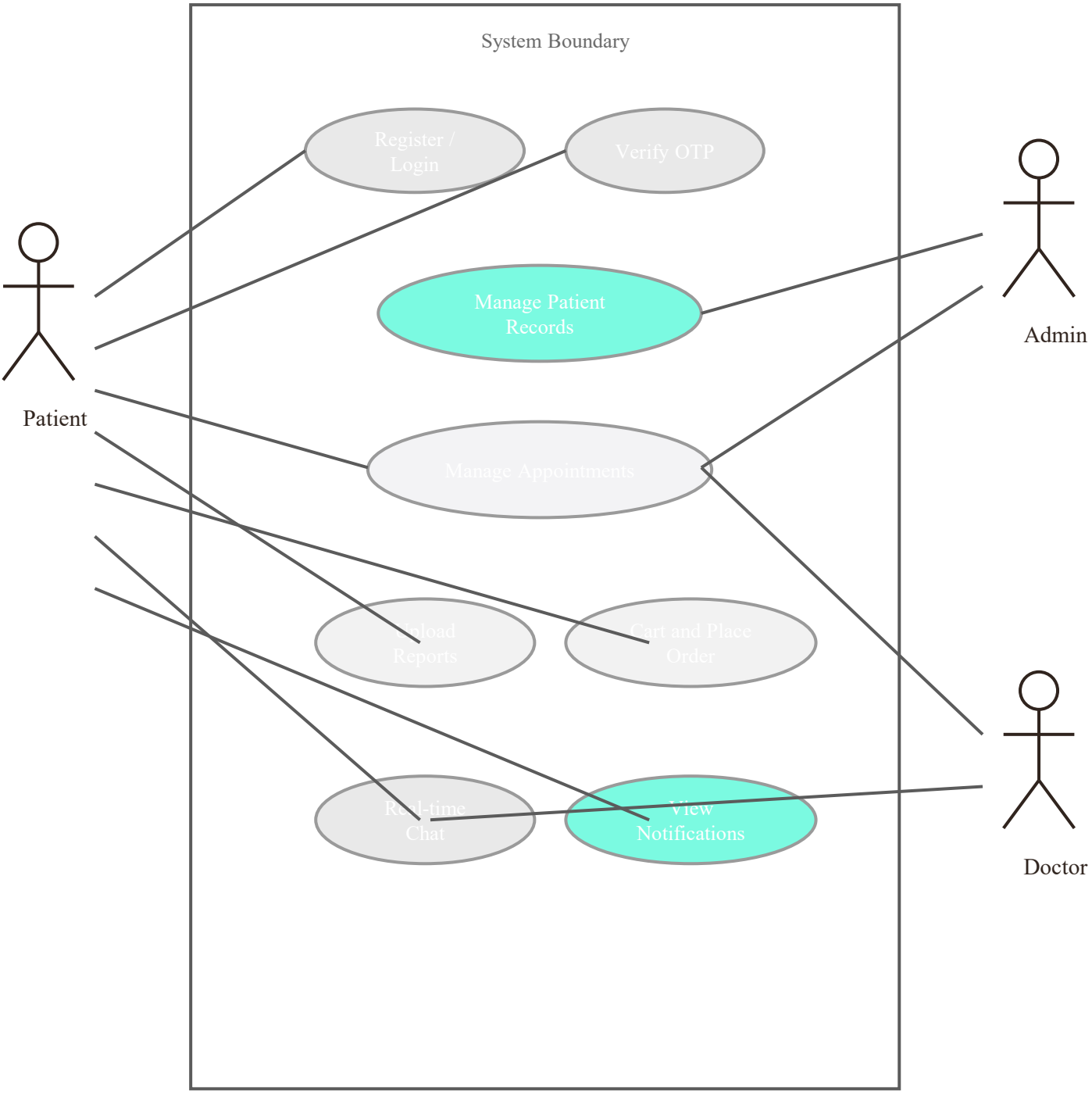

Figure 6. Use case diagram showing interactions among patient, admin, doctor, and the system.

4G. SEQUENCE DIAGRAM - APPOINTMENT BOOKING

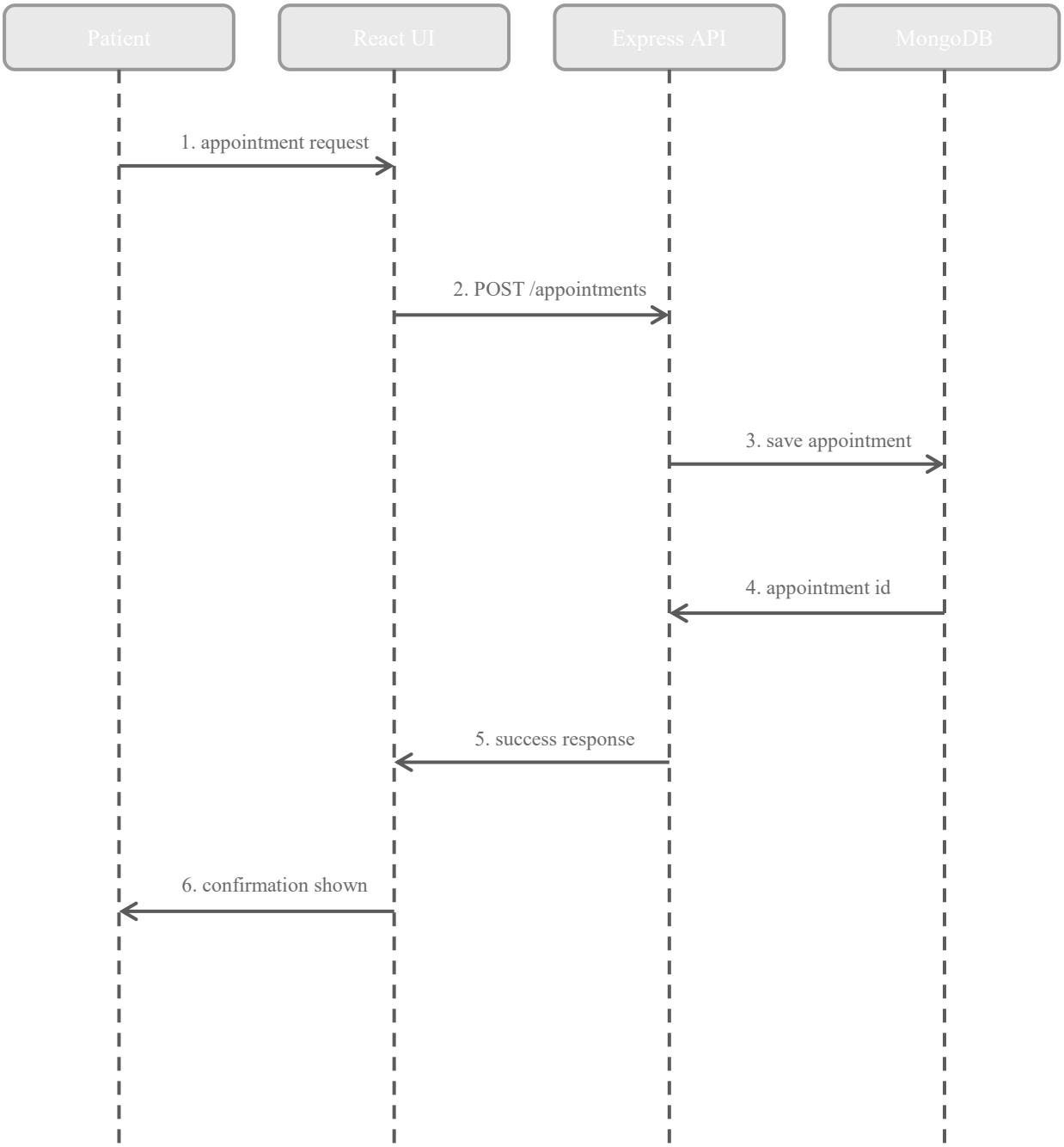

Figure 7. Sequence of interactions when an appointment is created through the web interface.

4H. SCHEMA / ENTITY RELATIONSHIP DIAGRAM

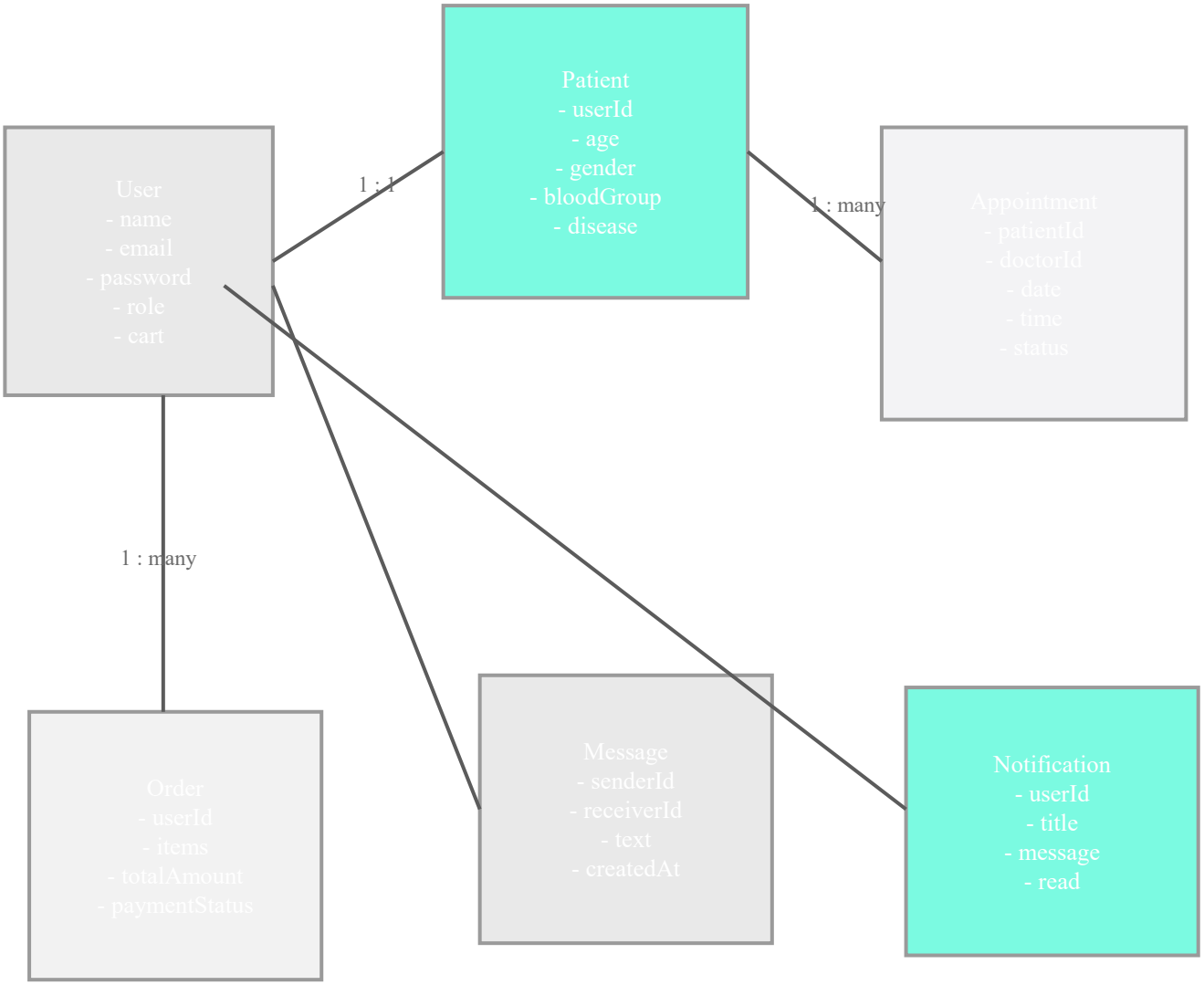

Figure 8. Simplified schema / ER view of the core MongoDB collections.

4I. FLOWCHART - AUTHENTICATION

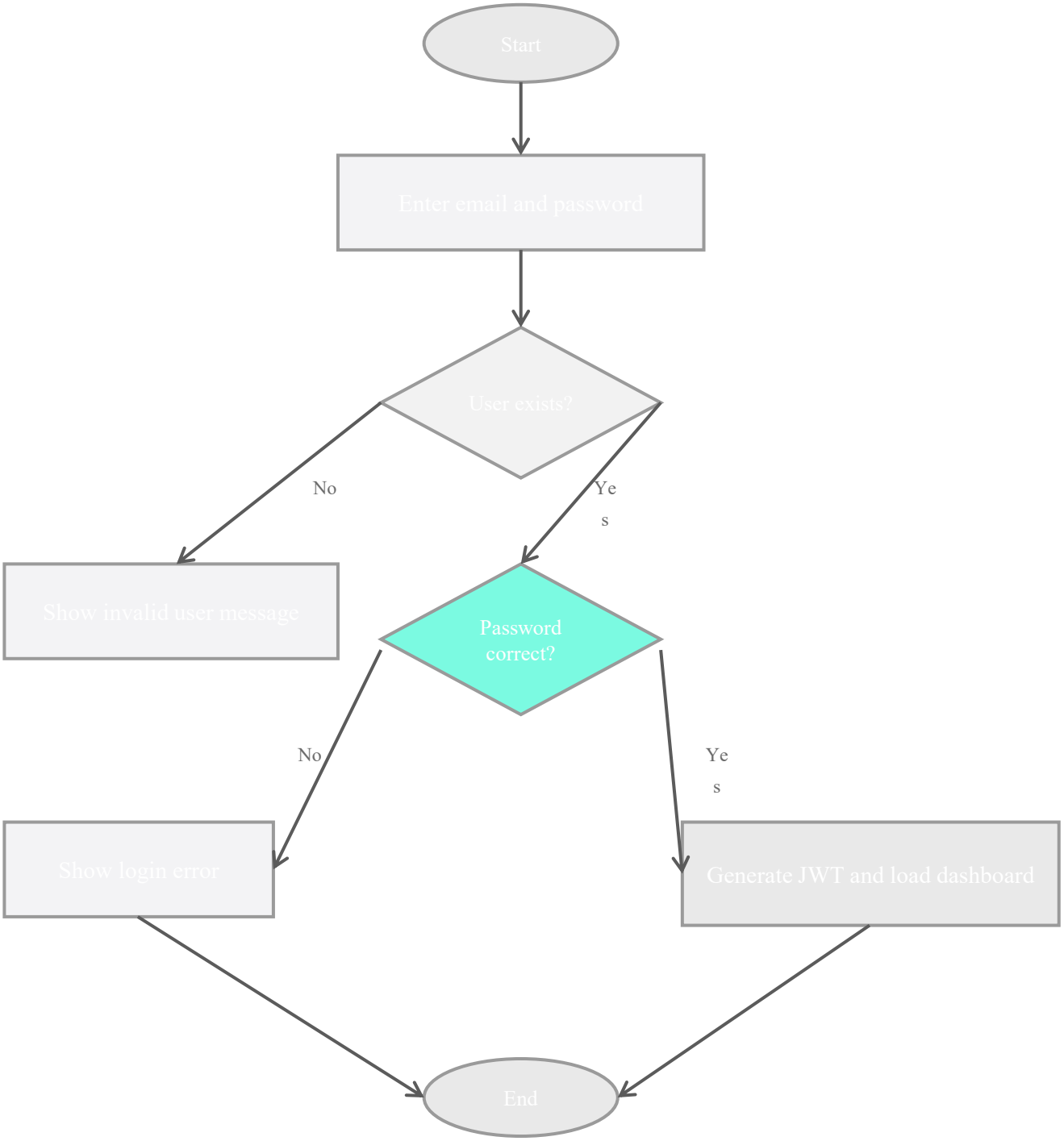

Figure 9. Flowchart of the login authentication process.

4J. FLOWCHART - ORDER PROCESSING

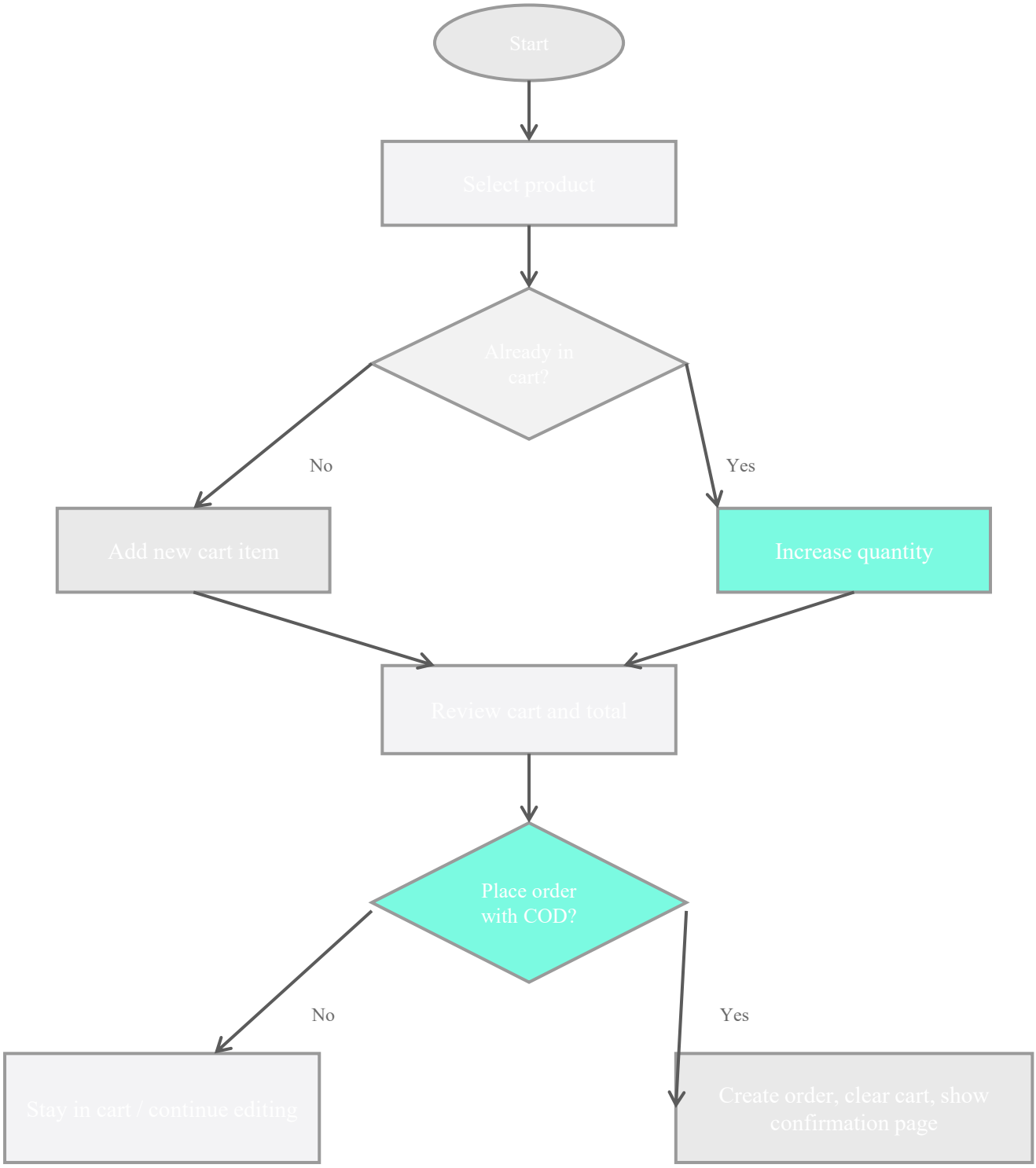

Figure 10. Flowchart of the cart and COD order processing logic.

CHAPTER 5: METHODOLOGY

The methodology adopted for this project is modular, incremental, and implementation-oriented. Each major function was developed as an independent module and then integrated into the broader workflow. This approach reduced complexity, simplified testing, and helped maintain code clarity during the project lifecycle.

This chapter presents the step-by-step development approach, the algorithms and techniques used, the simplified mathematical model considered for evaluation, and the overall workflow logic followed during execution.

5A. STEP-BY-STEP DEVELOPMENT APPROACH

- Requirement gathering from the problem statement, feature list, and report guidelines.
- Creation of backend project structure with config, models, controllers, routes, middleware, and utilities.
- Design of MongoDB schemas for users, patients, appointments, orders, messages, and notifications.
- Implementation of authentication, JWT protection, and OTP verification workflow.
- Addition of patient CRUD, appointment management, file upload, and search features.
- Implementation of medical store, cart logic, order placement, and order confirmation flow.
- Integration of Socket.IO chat, notifications, map, and AI doctor widget.
- Frontend design using React components, route protection, and role-aware dashboard sections.
- Testing using dummy accounts, seeded records, and manual validation of each workflow.

5B. ALGORITHMS AND TECHNIQUES USED

- Password Hashing: bcrypt is used to hash user passwords before storage.
- Token-based Authentication: JWT is generated at login and validated on protected routes.
- OTP Verification: a six-digit OTP with expiry timestamp is generated and matched during verification.
- Search and Filter: keyword matching and status-based filtering reduce retrieval time.
- Cart Quantity Merge: when the same product is added again, quantity is incremented instead of creating duplicate rows.
- Remove One Item Logic: cart item removal reduces quantity by one and removes the item only when quantity reaches zero.
- Delivered Order Removal Rule: order history deletion is permitted only after status becomes delivered.
- Real-time Messaging: Socket.IO emits events for instant message delivery.
- Role-based Authorization: middleware restricts admin-only functionality.

5C. MATHEMATICAL MODEL AND ASSUMPTIONS

Although the project is primarily application-oriented, a simplified performance model was used to reason about system effectiveness. Operational success can be expressed as a weighted function of authentication reliability, appointment workflow completion, order completion, and notification delivery.

A simple evaluation expression may be written as: Overall Success Score = w1(A) + w2(P) + w3(O) + w4(N), where A is authentication success rate, P is appointment flow completion rate, O is order flow completion rate, and N is notification or communication reliability. The weights reflect module importance within the prototype.

For report discussion, the following assumptions were used: one local deployment environment, seeded demo data, small to medium request volume, and manual validation of key workflows. These assumptions are appropriate for academic prototype evaluation, but not intended as enterprise-scale benchmarks.

5D. WORKFLOW EXPLANATION

The workflow begins with user onboarding. After account creation and login, the user enters the dashboard where accessible modules depend on the assigned role. The administrator manages patient information and appointments, while the patient primarily interacts with bookings, uploads, cart, orders, chat, and notifications.

The system is event-driven at the operational level. A new appointment triggers visibility and notifications; a new order triggers cart clearing and confirmation; a new chat message triggers an immediate socket event; and a delivered order unlocks a controlled delete action in history. Each workflow is designed so that state changes are reflected in both the frontend and backend records.

This methodology keeps the application behavior predictable and testable while still demonstrating practical full stack integration.

CHAPTER 6: IMPLEMENTATION

Implementation was carried out as a full stack MERN application with a clear separation between frontend and backend responsibilities. The backend exposes secure REST endpoints and socket events, while the frontend consumes those services through React pages and components.

This chapter documents the tools used, the hardware and software environment, the database and dummy data setup, the coding structure, the API organization, and the representative user interface layouts of the implemented system.

6A. TOOLS AND TECHNOLOGIES

Technology	Purpose in Project	Reason for Use
React.js	Frontend UI	Reusable components and structured client rendering
Node.js	Backend runtime	Single language across stack and non-blocking operations
Express.js	REST API framework	Lightweight routing and middleware support
MongoDB / Mongoose	Database and ODM	Flexible document storage for hospital records
Socket.IO	Real-time communication	Instant chat delivery between users
Multer	File upload handling	Simple report upload processing
JWT / bcrypt	Security	Token auth and hashed passwords
React Leaflet	Map integration	Interactive hospital location display
Groq API	AI doctor assistant	Symptom guidance through floating assistant widget

Table 5. Tools and technologies used in the final implementation.

6B. HARDWARE REQUIREMENTS

Component	Minimum Requirement	Recommended
Processor	Dual-core CPU	Modern multi-core processor
RAM	4 GB	8 GB or higher
Storage	10 GB free space	SSD with additional margin
Network	Internet for package setup and map tiles	Stable broadband connection
Display	1366 x 768	Full HD for easier development

The project was designed to run on a regular student development laptop. Because the application is browser-based and the database workload is moderate, the hardware requirements remain modest. However, smoother execution is achieved with 8 GB RAM, especially when the backend server, frontend development server, MongoDB, and browser are active at the same time.

Table 6. Hardware requirements for development and local execution.

6C. SOFTWARE REQUIREMENTS AND DUMMY DATA

Software	Version / Example	Usage
Operating System	Windows 10/11 or similar	Local development
Node.js	v18 or above	Runtime for backend and frontend tooling
npm	v9 or above	Package management
MongoDB	Community edition / local URI	Persistent storage
VS Code	Any recent version	Code editing and debugging
Browser	Chrome / Edge	Application testing

The system also uses seeded dummy data for demonstration. Example accounts include one administrator, one doctor, and one patient. Representative medical store items, patient disease details, sample appointments, and notification records allow all modules to be demonstrated without requiring external data collection.

Because the project is prototype-driven, the dummy data plays the role of a small dataset for functional validation rather than statistical analysis.

Table 7. Software requirements and the role of dummy data in testing.

6D. CODING APPROACH AND MVC STRUCTURE

The backend code is structured according to MVC principles. Models define the User, Patient, Appointment, Order, Message, and Notification schemas. Controllers contain business logic for authentication, patients, appointments, orders, chat, notifications, uploads, and AI assistance. Route files map API endpoints to the correct controller methods while middleware protects secure paths.

The frontend uses React pages and components to render dashboard sections such as Overview, Patients, Appointments, Orders, Cart, Upload, Map, Chat, and Notifications. Shared state, especially authenticated user information, is managed through context so that role-based behavior can be enforced at the interface level.

This coding approach keeps the project simple to understand while still following maintainable engineering practices.

6E. BACKEND MODULES AND API DESIGN

Module	Representative Endpoint	Responsibility
Authentication	/api/auth/login	Login, token generation, OTP workflow
Patients	/api/patients	CRUD operations on patient records
Appointments	/api/appointments	Create and update appointments
Orders	/api/orders	Cart actions, place order, delivery status
Upload	/api/upload	Store report files and return path
Chat	/api/chat/messages	Persist and fetch conversations
Notifications	/api/notifications	List and mark notifications
AI Doctor	/api/ai/doctor	Send symptom prompt to Groq API

Table 8. Core API summary of the implemented backend modules.

6F. FRONTEND IMPLEMENTATION AND REPRESENTATIVE SCREENS

The frontend is organized as reusable React components and clean role-based pages. The representative layouts above correspond to the major interaction surfaces in the implemented website.

Figure 11. Representative interface layouts aligned with the implemented login, dashboard, cart, confirmation, chat, and AI sections.

6G. SECURITY, VALIDATION, AND FILE HANDLING

- Passwords are hashed using bcrypt before storage in MongoDB.
- JWT middleware protects routes that require authenticated access.
- Role-based authorization restricts admin-only operations such as patient management and order delivery updates.
- OTP generation stores a temporary code and expiry timestamp for verification flow.
- Input validation checks required fields in authentication, appointment, and order forms.
- Multer processes multipart form data and stores uploads in a controlled server directory.
- Cart logic avoids duplicate product rows by increasing quantity when the same item is added again.
- Delivered order deletion is validated by status so users cannot remove active order history prematurely.

CHAPTER 7: RESULTS AND DISCUSSION

Results and discussion evaluate whether the implemented system meets the project objectives in a functional and operational sense. Because the project is a working prototype rather than a benchmarked production deployment, the emphasis is placed on workflow completion, stability, response quality, and user-visible outcomes.

Testing confirmed that the main modules including authentication, CRUD, appointments, file upload, search, cart, order confirmation, chat, notifications, and delivered-order controls behave as expected.

7A. TESTING STRATEGY AND TEST CASES

TC	Feature	Expected Result	Status
TC-01	Registration and login	User account created and authenticated	Pass
TC-02	OTP verification	Valid OTP marks user as verified	Pass
TC-03	Patient CRUD	Admin creates, edits, and deletes records	Pass
TC-04	Appointment booking	Appointment saved with status	Pass
TC-05	Search and filter	Matching records are returned	Pass
TC-06	Cart quantity merge	Same product increases quantity	Pass
TC-07	Order confirmation page	Total and confirmation view displayed	Pass
TC-08	Delivered order delete rule	Only delivered orders are removable	Pass
TC-09	File upload	File path returned and stored	Pass
TC-10	Chat and notifications	Messages and alerts are delivered	Pass

Table 9. Representative system test cases executed during validation.

7B. OUTPUT RESULTS AND MODULE-WISE OUTCOMES

- Authentication output: secure login, token issue, and OTP verification response.
- Admin output: patient creation and appointment administration through dashboard controls.
- Patient output: access to appointments, uploads, cart, order history, chat, and AI doctor support.
- Order output: a separate confirmation page displays order placed message, total amount, items, and continue buying action.
- Chat output: live message updates arrive without page refresh through Socket.IO.
- Notification output: users receive system alerts for order and appointment changes.
- UI output: improved visual layout with section cards, hero area, cleaner forms, and clearer navigation.

Taken together, these results demonstrate that the application satisfies the required feature set in a coordinated manner rather than as disconnected demos.

7C. PERFORMANCE METRICS AND EXPECTED VS ACTUAL COMPARISON

Metric	Expected	Observed in Local Demo
Login flow	Secure session creation	Successful with token and route protection
Search behavior	Fast record filtering	Near-instant on seeded dataset
Chat latency	Real-time delivery	Immediate in local environment
Order workflow	Cart to confirmation transition	Completed correctly with total calculation
Notification workflow	Automatic alert generation	Working for tested event triggers

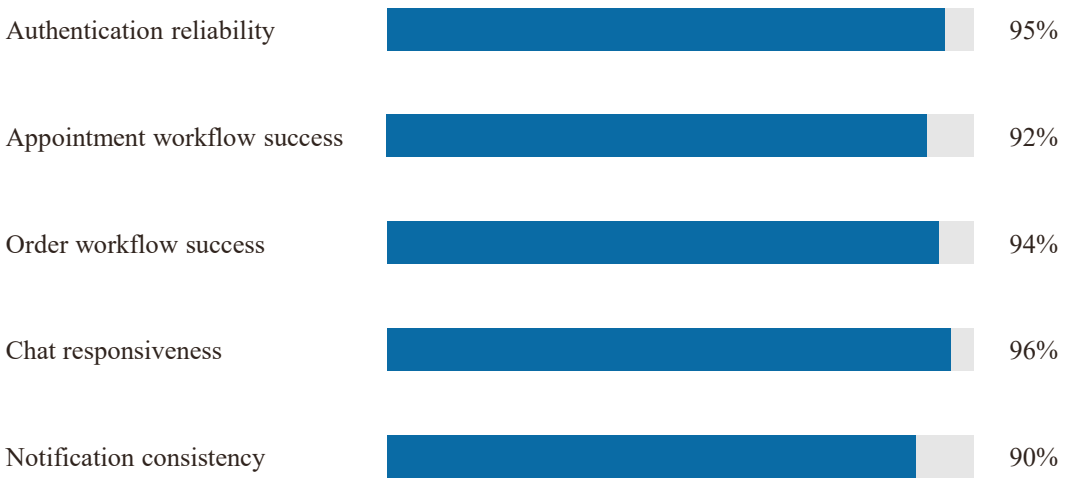

Table 10. Expected versus actual comparison with representative local-demo metrics.

7D. INTERPRETATION OF RESULTS

The observed results indicate that the system fulfills its prototype objectives effectively. The strongest outcomes are visible in integration and workflow continuity: patient data, appointments, uploads, orders, notifications, and communication are all handled from one interface. This is a significant improvement over fragmented manual processes.

The cart logic and confirmation page enhance the order module by making it behave like a realistic transactional flow. Similarly, the delivered-order deletion rule introduces business logic that prevents premature data removal. The chat and notification modules improve coordination, while the AI doctor widget demonstrates how a future-facing feature can be introduced without disrupting the core architecture.

Overall, the results suggest that the project is functionally successful, academically robust, and well-positioned for future expansion.

CHAPTER 8: ADVANTAGES AND LIMITATIONS

Every applied system has strengths and constraints. In this project, the advantages arise from integration, usability, and structured design, while the limitations mostly relate to prototype scope and the absence of advanced enterprise features.

Recognizing both aspects is important because it clarifies what the current project achieves and where future work should be directed.

8A. ADVANTAGES OF THE PROPOSED SYSTEM

- Centralizes hospital records, appointments, orders, and communication into one platform.
- Reduces paperwork and lowers the chance of manual record loss or duplication.
- Improves visibility of appointments and operational actions through structured dashboards.
- Supports direct interaction between users through real-time chat and notifications.
- Provides role-based access control and secure authentication practices.
- Demonstrates maintainable full stack design using MERN and MVC.
- Adds practical usability features such as cart quantity management, order confirmation, and document upload.
- Creates a foundation for advanced extensions such as telemedicine and AI-based support.

8B. LIMITATIONS AND CONSTRAINTS

- Online payment gateways are not fully integrated; the current order flow uses COD only.
- Uploaded files are stored locally rather than in cloud storage.
- The UI is improved but still intentionally simple for academic clarity rather than enterprise polish.
- The AI doctor assistant provides informational guidance and not a clinical diagnosis.
- Video consultation, wearable device linkage, and advanced analytics are not implemented in the current version.
- Testing is based on local prototype execution and seeded dummy data rather than production deployment.

These limitations do not reduce the academic validity of the project; instead, they define the boundary of the current prototype and identify the areas most suitable for future enhancement.

CHAPTER 9: CONCLUSION

The Smart Hospital Management System successfully addresses the problem of manual hospital record management by providing a centralized, role-based, and full stack digital solution. The project demonstrates how MERN technologies can be combined with MVC architecture to implement a secure and modular application for hospital operations.

The final system includes authentication, OTP verification, admin CRUD operations, patient and appointment management, report uploads, order processing with cart and COD, real-time chat, notifications, map integration, and an AI doctor assistant. Each of these modules contributes to a more connected and user-friendly workflow than the existing manual system.

From an academic perspective, the project satisfies the major requirements of analysis, design, implementation, testing, and discussion. From a practical perspective, it establishes a strong prototype that can be further expanded into a more complete hospital management ecosystem.

CHAPTER 10: FUTURE SCOPE

Enhancement Area	Possible Extension	Expected Benefit
Telemedicine	Video consultation and remote follow-up	Improved accessibility for patients
Wearable Integration	Vital sign tracking through health devices	Continuous monitoring and richer data
Payments	UPI or gateway integration	Complete digital billing experience
Cloud Storage	Document storage on cloud platforms	Scalable and secure report handling
Analytics	Charts and trend reports for admin	Operational decision support
Mobile App	React Native companion application	Broader access across devices

Future work may also include multilingual support, prescription PDF generation, doctor shift planning, and richer AI-assisted triage workflows. Because the current codebase is modular, these enhancements can be added progressively without requiring a complete redesign.

Table 11. Future enhancement roadmap for the proposed system.

REFERENCES

[1] A. R. Bakker and J. L. Mol, ""Hospital information systems,"" Eff Health Care, vol. 1, no. 4, pp. 215-223, 1983.

[2] P. L. Reichertz, ""Hospital information systems - past, present, future,"" Int J Med Inform, vol. 75, no. 3-4, pp. 282-299, 2006.

[3] C. A. Bain and C. Standing, ""A technology ecosystem perspective on hospital management information systems: lessons from the health literature,"" Int J Electron Healthc, vol. 5, no. 2, pp. 193-210, 2009.

[4] J. Sligo, R. Gauld, V. Roberts, and L. Villa, ""A literature review for large-scale health information system project planning, implementation and evaluation,"" Int J Med Inform, vol. 97, pp. 86-97, 2017.

[5] P. W. Handayani, A. N. Hidayanto, and I. Budi, ""User acceptance factors of hospital information systems and related technologies: Systematic review,"" Inform Health Soc Care, vol. 43, no. 4, pp. 401-426, 2018.

[6] M. Khalifa and O. Alswailem, ""Hospital information systems (HIS) acceptance and satisfaction: a case study of a tertiary care hospital,"" Procedia Comput Sci, vol. 63, pp. 198-204, 2015.

REFERENCES (CONTINUED)

[7] J. S. McCullough, ""The adoption of hospital information systems,"" Health Econ, vol. 17, no. 5, pp. 649-664, 2008.

[8] MongoDB, ""MongoDB Documentation."" [Online]. Available: https://www.mongodb.com/docs/ . [Accessed: Apr. 18, 2026].

[9] Express.js, ""Express Documentation."" [Online]. Available: https://expressjs.com/ . [Accessed: Apr. 18, 2026].

[10] React, ""React Documentation."" [Online]. Available: https://react.dev/ . [Accessed: Apr. 18, 2026].

[11] Node.js, ""Node.js Documentation."" [Online]. Available: https://nodejs.org/en/docs . [Accessed: Apr. 18, 2026].

[12] Socket.IO, ""Socket.IO Documentation."" [Online]. Available: https://socket.io/docs/v4/ . [Accessed: Apr. 18, 2026].

[13] Mongoose, ""Mongoose Documentation."" [Online]. Available: https://mongoosejs.com/docs/ . [Accessed: Apr. 18, 2026].

[14] IETF, ""RFC 7519: JSON Web Token (JWT)."" [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519 . [Accessed: Apr. 18, 2026].

[15] Leaflet, ""Leaflet Documentation."" [Online]. Available: https://leafletjs.com/ . [Accessed: Apr. 18, 2026].

APPENDIX A: SAMPLE API ENDPOINTS

POST	/api/auth/register	Create a new user account
POST	/api/auth/login	Authenticate and issue JWT
POST	/api/auth/send-otp	Generate and send email OTP
POST	/api/auth/verify-otp	Verify OTP and mark user verified
GET	/api/patients	Fetch patient records
POST	/api/patients	Create patient profile
PUT	/api/patients/:id	Update patient information
DELETE	/api/patients/:id	Delete patient record
POST	/api/appointments	Create appointment
PUT	/api/appointments/:id/status	Update appointment status
POST	/api/orders/cart	Add item to cart
PATCH	/api/orders/cart/remove-one	Remove one quantity from cart
POST	/api/orders/place	Place order with COD
DELETE	/api/orders/:id	Delete delivered order history entry
POST	/api/upload	Upload report file
GET	/api/notifications	List notifications
POST	/api/ai/doctor	Symptom prompt to AI doctor assistant

APPENDIX B: DUMMY DATA SAMPLES

Representative demo accounts used for testing include: Admin - admin@hospital.com / 123456, Doctor - doctor@hospital.com / 123456, and Patient - patient@hospital.com / 123456. Dummy patient examples include records such as Rahul Verma and other manually added profiles from the admin dashboard.

Sample medical store entries include tablet, syrup, and report-related consumables with stored price and quantity data. Example appointment states include Scheduled, Completed, and Cancelled. Notifications are generated on order placement, appointment updates, and welcome or system events.

This appendix confirms that the system was designed to be demonstrable without external datasets while still preserving realistic hospital management behavior.

- Cart behavior: repeated product addition increments quantity.
- Cart removal behavior: remove action decreases one quantity at a time.
- Order history behavior: deletion is allowed only when an order is marked delivered.
- Confirmation behavior: placing COD order redirects to a confirmation page with total amount and continue buying action.

APPENDIX C: SOURCE CODE ORGANIZATION

```
backend/
  config/
  controllers/
  middleware/
  models/
  routes/
  seed/
  uploads/
  utils/
  server.js

frontend/
  src/
    components/
    context/
    pages/
    services/
    styles/
  App.jsx

report/

Smart_Hospital_Management_System_Standards_Report.pdf

Smart_Hospital_Management_System_Standards_Report.pptx
  diagrams/
```

The report diagrams have also been exported separately to the diagrams folder for easy reuse in presentations or document editing. This appendix gives a compact overview of the implemented codebase and the generated report assets.

- 01_system_architecture.png
- 03_dfd_level_0.png
- 04_dfd_level_1_patient.png
- 05_dfd_level_1_admin_doctor.png
- 06_use_case_diagram.png
- 07_sequence_diagram.png